Code Verification in Rust with Verus

Matt Grimes

Oct 30 2026

What is code verification

Z3

Verus

Verification as LLM Guardrails

What is code verification

What is code verification

Z3

Verus

Verification as LLM Guardrails

What does it mean for code to be "correct?"

To ask if code is "correct," we have to formulate a formal claim about what the code is supposed to do.

Taking a max

If *max* returns the max of two integers, x and y, we expect the following properties:

- $max(x, y) \geq x$
- $max(x, y) \geq y$
- $max(x, y) \in \{x, y\}$

Taking the max of two integers

We can state the logical claim that the implementation is correct as

$$\forall x, y, m \in \mathbb{Z},\ m = \max(x, y) \implies (m = x \vee m = y) \wedge m \geq x \wedge m \geq y.$$

Taking the max of two integers

We can state the logical claim that the implementation is correct as

$$\forall x, y, m \in \mathbb{Z},\ m = \max(x, y) \implies (m = x \lor m = y) \land m \geq x \land m \geq y.$$

In general, $a \implies b$ is equivalent to $\neg a \lor b$. We can thus rewrite as

$$\forall x, y, m \in \mathbb{Z},\ \neg(m = \max(x, y)) \lor ((m = x \lor m = y) \land m \geq x \land m \geq y).$$

Taking the max of two integers

We can state the logical claim that the implementation is correct as

$$\forall x, y, m \in \mathbb{Z},\ m = \max(x, y) \implies (m = x \vee m = y) \wedge m \geq x \wedge m \geq y.$$

In general, $a \implies b$ is equivalent to $\neg a \vee b$. We can thus rewrite as

$$\forall x, y, m \in \mathbb{Z},\ \neg(m = \max(x, y)) \vee ((m = x \vee m = y) \wedge m \geq x \wedge m \geq y)\,.$$

Equivalently, the implementation of max has a bug if and only if

$$\exists x, y, m \in \mathbb{Z},\ (m = \max(x, y)) \wedge \neg((m = x \vee m = y) \wedge m \geq x \wedge m \geq y)\,.$$

What about the implementation?

In pseudo-code, we can define a naive implementation of max:

```
fn max(x: int, y: int) -> (m: int)
{
    if x > y
    {
        return x;
    } else
    {
        return y;
    }

}
```

Logically, this implementation can be expressed by

$$m = \max(x, y) := ((x > y) \wedge (m = x)) \vee (\neg(x > y) \wedge (m = y))$$

Putting it all together

We can combine everything on the prior slides into the following statement: our implementation of max is *incorrect* if and only if $\exists x, y, m \in \mathbb{Z}$ such that

$$\underbrace{[((x > y) \wedge (m = x)) \vee (\neg(x > y) \wedge (m = y))]}_{m \text{ given by } \max(x,y)}$$

Putting it all together

We can combine everything on the prior slides into the following statement: our implementation of max is *incorrect* if and only if $\exists x, y, m \in \mathbb{Z}$ such that

$$\underbrace{[((x > y) \wedge (m = x)) \vee (\neg(x > y) \wedge (m = y))]}_{m \text{ given by } \max(x,y)} \wedge \neg \underbrace{[(m = x \vee m = y) \wedge m \geq x \wedge m \geq y]}_{m \text{ satisfies definition of max}}.$$

Putting it all together

We can combine everything on the prior slides into the following statement: our implementation of max is *incorrect* if and only if $\exists x, y, m \in \mathbb{Z}$ such that

$$\underbrace{[((x > y) \wedge (m = x)) \vee (\neg(x > y) \wedge (m = y))]}_{m \text{ given by } \max(x,y)} \wedge \neg \underbrace{[(m = x \vee m = y) \wedge m \geq x \wedge m \geq y]}_{m \text{ satisfies definition of max}}.$$

We can see by inspection that the claim is false, there is no counter-example, and the implementation is proven correct.

Z3

What is code verification

Z3

Verus

Verification as LLM Guardrails

Z3: satisfiability relative to a theory

In the last section, we asked whether or not $\exists x, y, m \in \mathbb{Z}$ such that

$$\underbrace{[((x > y) \wedge (m = x)) \vee (\neg(x > y) \wedge (m = y))]}_{m \text{ given by } \max(x,y)} \wedge \neg \underbrace{[(m = x \vee m = y) \wedge m \geq x \wedge m \geq y]}_{m \text{ satisfies definition of max}}$$

This is exactly the type of statement the SMT/SAT solver Z3 is designed to answer.

SMT: satisfiability modulo theories

Given a formula and a background "theory", is the formula satisfiable?

- Boolean combinations of constraints in supported theories.
- Examples: integer arithmetic, fixed-width bit-vectors, arrays.
- The theory determines the meaning of symbols such as $+$ and $\leq$.

For our example: linear constraints over the integers.

Source: Microsoft's Z3 Guide

Prove a property by searching for its violation

Ask whether the existential counterexample formula is satisfiable.



- `sat`: a violation exists in this encoding.
- `unsat`: no such violation exists in this encoding.
- `unknown`: the solver could not decide; no proof obtained.

Source: Z3 Guide: Basic Commands

Prove a property by searching for its violation

Ask whether the existential counterexample formula is satisfiable.



- `sat`: a violation exists in this encoding.
- `unsat`: no such violation exists in this encoding.
- `unknown`: the solver could not decide; no proof obtained.

Source: Z3 Guide: Basic Commands

A timeout also leaves the obligation unproved. With abstractions, a satisfying model may require interpretation before calling it a real program counterexample. Our small exact arithmetic example has a direct interpretation.

Ask Z3 to find any bugs

```
(define-fun is_max ((m Int) (x Int) (y Int)) Bool
    (and
        (>= m x)
        (>= m y)
        (or
            (= m x)
            (= m y)
        )
    )
)
(define-fun max ((x Int) (y Int)) Int
    (ite (> x y) x y)
)

(declare-const x Int)
(declare-const y Int)
(declare-const m Int)
(assert
    (and
        (= m (max x y))
        (not (is_max m x y))
    )
)
(check-sat)
(get-model)
```

Result: `unsat`

Verus

What is code verification

Z3

Verus

Verification as LLM Guardrails

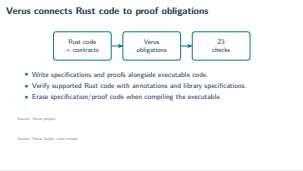
Verus connects Rust code to proof obligations



- Write specifications and proofs alongside executable code.
- Verify supported Rust code with annotations and library specifications.
- Erase specification/proof code when compiling the executable.

Source: Verus project

Source: Verus Guide: code modes



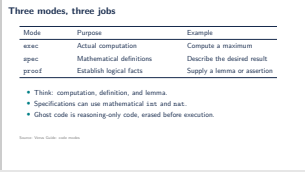
Verus supports a subset of Rust; existing crates are not automatically fully verified. The diagram is conceptual, not a complete description of the internal compiler pipeline.

Three modes, three jobs

Mode	Purpose	Example
`exec`	Actual computation	Compute a maximum
`spec`	Mathematical definitions	Describe the desired result
`proof`	Establish logical facts	Supply a lemma or assertion

- Think: computation, definition, and lemma.
- Specifications can use mathematical `int` and `nat`.
- Ghost code is reasoning-only code, erased before execution.

Source: Verus Guide: code modes



Machine integers still matter in exec code. Mathematical types let the specification describe arithmetic without silently wrapping. Proof functions can package reusable arguments.

Implementing max in Rust

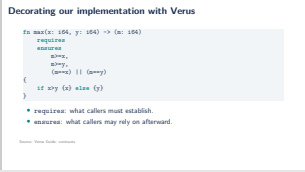
```
fn max(x: i64, y: i64) -> i64
{
    if x>y {x} else {y}
}
```

Decorating our implementation with Verus

```
fn max(x: i64, y: i64) -> (m: i64)
    requires
    ensures
        m>=x,
        m>=y,
        (m==x) || (m==y)
{
    if x>y {x} else {y}
}
```

- `requires`: what callers must establish.
- `ensures`: what callers may rely on afterward.

Source: Verus Guide: contracts



The complete file is examples/max.rs. There are no preconditions on the two i64 inputs. The executable function only compares and selects, so there is no arithmetic-overflow obligation for its body. The three postconditions characterize the maximum.

Using a verified function is applying a lemma

Let $M(a, b)$ denote the verified function. Its contract gives

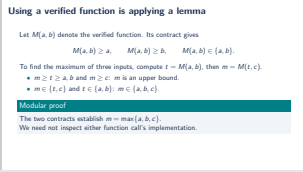
$$M(a, b) \geq a, \qquad M(a, b) \geq b, \qquad M(a, b) \in \{a, b\}.$$

To find the maximum of three inputs, compute $t = M(a, b)$, then $m = M(t, c)$.

- $m \geq t \geq a, b$ and $m \geq c$: m is an upper bound.
- $m \in \{t, c\}$ and $t \in \{a, b\}$: $m \in \{a, b, c\}$.

Modular proof

The two contracts establish $m = \max\{a, b, c\}$.
We need not inspect either function call's implementation.

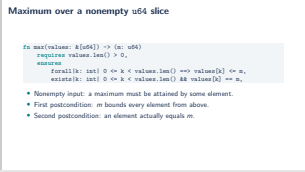


All inputs are representable machine integers; comparison and selection stay in the same type. This is ordinary lemma application followed by transitivity. Repeating this argument suggests the maximum-over-a-slice algorithm on the next slides.

Maximum over a nonempty u64 slice

```
fn max(values: &[u64]) -> (m: u64)
    requires values.len() > 0,
    ensures
        forall|k: int| 0 <= k < values.len() ==> values[k] <= m,
        exists|k: int| 0 <= k < values.len() && values[k] == m,
```

- Nonempty input: a maximum must be attained by some element.
- First postcondition: *m* bounds every element from above.
- Second postcondition: an element actually equals *m*.



The binary example used i64. For this loop example we use u64 as requested; the same order argument works for both. A slice is a sequence viewed without copying it. The mathematical int indices in the quantifiers are ghost expressions, not runtime iteration.

What about loops?

Verus handles loops in rust by induction.
For each loop, we must establish that it

- terminates
- the induction hypothesis implies the next step

Proof idea

We assume `values` is non-empty, so use induction on the length of `values`.

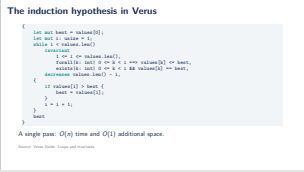
- If `values.len() == 1`, then the max is `values[0]`.
- For the induction hypothesis, assume that for all $k <$ `values.len()`, `values[0..k]` $\leq$ `values[j]`, for some j.
- We can then consider `values[k+1]`—if it's greater than the current max use it, otherwise stick with what we have.
- This process terminates because `values` is of finite length.

The induction hypothesis in Verus

```
{
    let mut best = values[0];
    let mut i: usize = 1;
    while i < values.len()
        invariant
            1 <= i <= values.len(),
            forall|k: int| 0 <= k < i ==> values[k] <= best,
            exists|k: int| 0 <= k < i && values[k] == best,
        decreases values.len() - i,
    {
        if values[i] > best {
            best = values[i];
        }
        i = i + 1;
    }
    best
}
```

A single pass: $O(n)$ time and $O(1)$ additional space.

Source: Verus Guide: Loops and invariants



This is the body of max from the preceding slide, read directly from examples/max_iter.rs. best represents b and values represents a. The loop starts at 1 because values[0] has already been processed. While i $<$ len, the next access is in bounds and i $+$ 1 cannot overflow usize. There is no arithmetic on the u64 elements. Verus checks the upper-bound and witness invariants, the postconditions, and the decreasing distance to the end.

What is being trusted?

- The specification captures the intended theorem.
- The translation from code to logical obligations is sound.
- The solver's result and any assumed lemmas are correct.
- Compilation and execution preserve the verified meaning.

Verification as LLM Guardrails

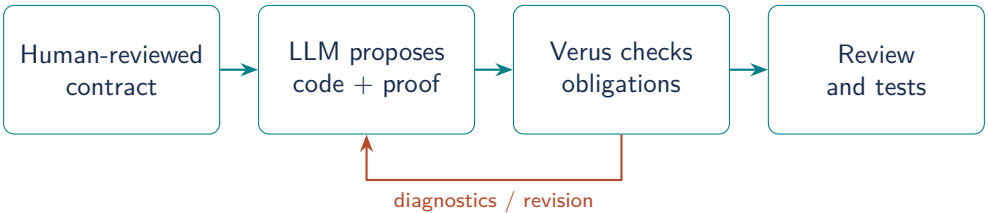
What is code verification

Z3

Verus

Verification as LLM Guardrails

An LLM proposes a construction; verification checks it



A concrete task: count equal pairs

Given a list s, count pairs of positions carrying the same value:

$$E(s) = \left|\{(i,j) : 0 \le i < j < |s|,\ s_i = s_j\}\right|.$$

For $s = [4,4,4,9,9]$, there are $\binom{3}{2} + \binom{2}{2} = 4$ pairs.

The recursive specification in `equal_pairs.rs`

$$E([]) = 0, \qquad E(p +\!\!+ [v]) = E(p) + |\{i : p_i = v\}|.$$

Appending v creates one pair with each earlier occurrence of v.

The Spec and Task

```
pub open spec fn spec_equal_pairs(s: Seq<u64>) -> nat
    decreases s.len()
{
    if s.len() == 0 {
        0
    } else {
        let prefix = s.drop_last();

        spec_equal_pairs(prefix)
            + prefix.filter(|x: u64| x == s.last()).len()
    }
}

fn equal_pairs(s: &[u64]) -> (r: u64)
    requires
        spec_equal_pairs(s@) <= u64::MAX
    ensures
        r == spec_equal_pairs(s@)
{
    <LLM can write here>
}
```

The guardrail has a defined reach

Can establish, when specified

- The result is an attained maximum.
- Slice accesses stay in bounds.
- The prefix invariant is preserved.

Still needs separate evidence

- The contract matches user intent.
- External systems match the model.
- The algorithm is efficient enough to use.

Stronger acceptance criteria make generated code easier to evaluate.

In Practice

1. Write the contract before polishing the implementation.
2. Gradually add contract components for existing code.
3. Make verification a required check for LLM-generated changes.

References and demo files

- This deck along with examples

  `https://github.com/matthewgrimes/verus-demo`

- Verus Tutorial and Reference

  `https://verus-lang.github.io/verus/guide/`

- Verus Tutorial and Reference

  `https://verus-lang.github.io/verus/guide/`

- Verus source and installation instructions

  `https://github.com/verus-lang/verus`

- Microsoft's Z3 Guide

  `https://microsoft.github.io/z3guide/`

Local examples: `examples/max.rs`, `examples/max_iter.rs`, `examples/max.smt2`, `examples/equal_pairs.rs`.